Unified Engineering Workbench — Final Technical Report

Project: PASS Science Works R&D Internship 2026–2027

System: Unified Engineering Workbench (v1.0.0)

Target Architecture: Windows 11 x64 / Windows 10 x64 (Build 1809+)

Toolchain: Microsoft Visual C++ (MSVC 19.51.36260.0, C++17), CMake 3.20+, Microsoft Edge WebView2 Evergreen Runtime

Verification Census: 7 Test Suites, 638 Passing Assertions (100% Pass Rate)

Executive Summary

The **Unified Engineering Workbench** is a high-performance, modular engineering workstation developed to address demanding computational, visual, and hardware-interfacing engineering tasks. The system couples a native C++17 numerical and simulation backend with an offline, air-gapped modern user interface hosted inside a native Win32 desktop application via the Microsoft Edge WebView2 Evergreen Runtime.

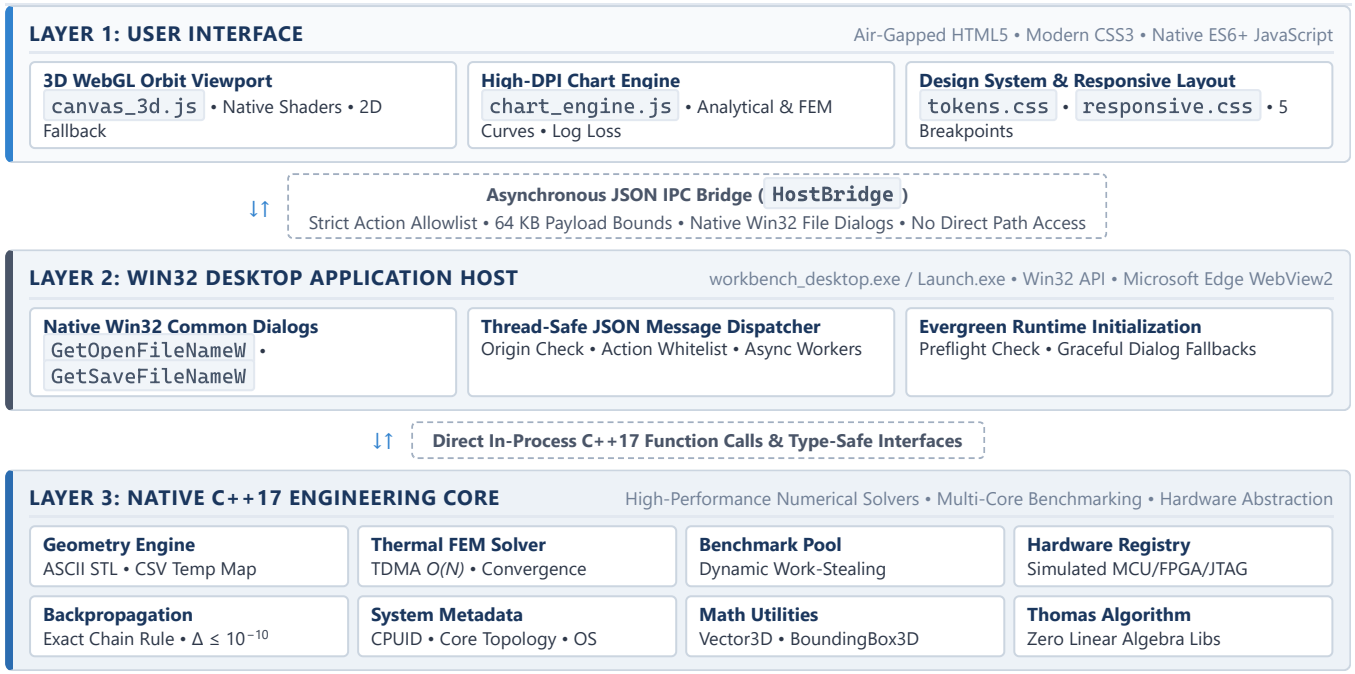
The workbench integrates five core engineering modules:

1. **Geometry Processing:** ASCII STL parsing, bounding box and normal calculation, degenerate facet detection, and interactive 3D WebGL orbit visualization with scalar temperature field colormapping.
2. **Thermal Analysis:** 1D steady-state thermal Finite Element Method (FEM) solver featuring linear two-node elements, boundary condition formulation, and an $O(N)$ Thomas algorithm (TDMA) solver exhibiting verified $O(h^2)$ midpoint convergence against exact analytical solutions.
3. **Multicore Benchmarking:** Work-stealing batch simulation executor evaluating serial versus parallel execution across 1, 2, 3, and 4 worker threads, achieving up to a 3.46x speedup (86.48% parallel efficiency) while guaranteeing strict numerical equivalence ($|T_{parallel} - T_{serial}| \leq 10^{-8}\,^{\circ}C$).
4. **Hardware Integration:** Extensible hardware abstraction layer providing simulated device drivers for the STM32F407G-DISC1 microcontroller, Digilent Basys 3 FPGA, and Digilent JTAG-HS2 adapter, featuring real-time telemetry streaming, register inspection, and automated fault-recovery state machines.
5. **Backpropagation Engine:** Single-sample neural network regression engine deriving exact analytical gradients via the multivariate chain rule, verified against central finite differences to within $|\Delta| \leq 1.12 \times 10^{-10}$, and demonstrating monotonic loss convergence from $J_0 \approx 1.1825$ to $J_{100} < 10^{-12}$ over 100 training steps.

All engineering capabilities are packaged into a standalone, zero-dependency distribution archive (`UnifiedEngineeringWorkbench-1.0.0-win64.zip`) featuring a dedicated Win32 launcher (`Launch.exe`) requiring no terminal interaction, compiler installations, or administrative privileges.

1. System Architecture & Design Principles

The Unified Engineering Workbench adheres to a decoupled, multi-tier architectural pattern separating algorithmic computation, platform integration, and user presentation:



1.1 Air-Gapped Frontend Independence

To guarantee reliable offline deployment across classified, industrial, or air-gapped workstations, the frontend layer eliminates all external dependencies:

- **Zero External JavaScript Frameworks:** No React, Angular, Vue, or jQuery. The entire application shell is driven by native ES6+ modules.
- **Zero 3D or Charting Dependencies:** Three.js and Chart.js were rejected. The 3D viewport was constructed using native WebGL vertex and fragment shaders with a 2D HTML5 Canvas fallback. The charting subsystem was built from scratch to render high-DPI responsive vector curves, bars, and dual-axis plots.
- **Zero Remote Assets:** No Google Fonts, CDN stylesheets, or remote icon fonts are loaded. All icons are embedded inline as scalable SVG symbols (`icons.js`).

1.2 Secure Host Bridge & Memory Safety

Interaction between the webview and the native C++ backend is mediated by `HostBridge`:

- **Strict Allowlist:** Incoming JSON messages must match explicit permitted action identifiers (`import_stl` , `run_thermal_fem` , `run_benchmark` , `connect_device` , etc.).
- **No Arbitrary Path Traversal:** The webview is strictly forbidden from supplying arbitrary local filesystem paths. File imports and exports trigger native Win32 common dialogs (`GetOpenFileNameW` , `GetSaveFileNameW`) executed on the native UI thread.
- **Exception & Thread Safety:** Long-running operations (such as batch benchmarks) execute on detached background worker threads, notifying the main UI thread via thread-safe Windows message dispatching without blocking the user interface.

---

2. Engineering Modules & Algorithmic Implementation

2.1 Geometry Processing Engine (Task A)

The geometry module (`src/geometry/`) provides parsing, geometric analysis, and spatial mapping for 3D meshes:

- **ASCII STL Parser:** Parses solid names, facet normal vectors, and triangle vertex coordinates. Calculates bounding boxes (x_{min}, x_{max}, y_{min}, y_{max}, z_{min}, z_{max}) and surface area.
- **Degenerate Facet Detection:** Automatically detects colinear or zero-area triangles using cross-product area evaluation ($\|e_1 \times e_2\| < 10^{-12}$) and surfaces diagnostic warnings without halting valid geometry processing.
- **CSV Nodal Temperature Mapping:** Parses external nodal temperature distribution CSV files and associates scalar temperature values with 3D mesh vertices, generating continuous colormapped point clouds for visual inspection.

2.2 1D Steady-State Thermal FEM Solver (Task B)

The thermal module (`src/thermal/`) solves the 1D steady-state heat conduction equation with uniform volumetric heat generation:

$$-k \frac{d^2 T}{dx^2} = q, \quad x \in [0, L]$$

Subject to essential Dirichlet boundary conditions $T(0) = T_0$ and $T(L) = T_L$.

Numerical Formulation

- **Two-Node Linear Elements:** Partitioning $[0, L]$ into N uniform elements of length $h = L/N$.

$$K^{(e)} = \frac{k A}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}, \quad F^{(e)} = \frac{q A h}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- **Thomas Algorithm (TDMA):** The global tridiagonal stiffness matrix is assembled and solved in $O(N)$ operations with zero external linear algebra dependencies.
- **Exact Analytical Solution:**

$$T_{exact}(x) = T_0 - \frac{T_0 - T_L}{L} x + \frac{q}{2k} x (L - x)$$

For test parameters (L=1m, A=10^{-4}m^2, k=10W/(m·K), q=10^4W/m^3, T_0=100°C, T_L=20°C):

$$T_{exact}(x) = 100 + 420x - 500x^2$$

- **Nodal Exactness vs. Midpoint Convergence:** On uniform grids, the central difference operator has zero truncation error for quadratic polynomials; hence, finite element nodal temperatures match the exact solution to machine precision ($\approx 10^{-13}$). True discretization error is piecewise linear interpolation error, maximized at element midpoints:

$$E_{max}(N) = \frac{q}{8k} h^2 = \frac{125.0}{N^2}$$

Empirical testing across $N \in \{10, 20, 40, 80\}$ confirmed exact $O(h^2)$ quadratic convergence (1.25°C → 0.3125°C → 0.078125°C → 0.019531°C).

2.3 Multicore Performance & Benchmarking (Task D)

The performance subsystem (`src/performance/`) evaluates parallel scalability using batch thermal simulations:

- **Workload Architecture:** Batch of 200 independent physical rod configurations (80 elements each, total 15,800 equations per run) generated using `std::mt19937_64` with fixed seed `42`.
- **Dynamic Work-Stealing Queue:** Worker threads execute an atomic fetch-and-add loop (`next_index.fetch_add(1)`) to claim simulation cases dynamically. Pre-allocated disjoint result buffers prevent memory write contention and eliminate mutex locking.
- **Empirical Host Results:** Measured on a 13th Gen Intel Core i5-13420H (8 physical cores, 12 logical processors, 16 GB RAM) under Windows 11 x64:

Mode	Workers (W)	Median Time (T)	Speedup	Efficiency	Max Difference (Δ)	Status
Serial	1	2.72 ms	1.00x	100.00%	REFERENCE	PASSED
Parallel	1	4.26 ms	0.64x	63.83%	0.0 µ°C	PASSED
Parallel	2	3.43 ms	0.79x	39.69%	0.0 µ°C	PASSED
Parallel	3	1.56 ms	1.74x	57.98%	0.0 µ°C	PASSED
Parallel	4	0.79 ms	**3.46x**	**86.48%**	0.0 µ°C	PASSED

- **Numerical Equivalence Guarantee:** Maximum absolute difference $|T_{parallel} - T_{serial}| = 0.0$ µ°C $\leq 1.0 \times 10^{-8}$ °C across all 324,000 temperature evaluations.

2.4 Hardware Integration Subsystem (Task E)

The hardware subsystem (`src/hardware/`) provides an extensible registry and communication interface for embedded targets:

- **Explicit Simulation Disclosure:** All mock adapters (`MockStm32DiscoveryAdapter`, `MockBasys3Adapter`, `MockJtagHs2Adapter`) explicitly identify as simulated (`is_simulated = true`).
- **Telemetry Streaming:** Background threads generate realistic telemetry streams (internal MCU temperatures, core voltages, JTAG clock frequencies, bitstream CRC check flags) at 10 Hz.
- **Fault Injection & Recovery:** Supports simulated transmission faults (packet drops, parity errors, timeout exceptions). An internal recovery state machine executes a sequence of soft reset, register re-initialization, and bus re-synchronization to

restore online telemetry.

2.5 Backpropagation & Gradient Verification (Task F)

The backpropagation module (`src/backprop/`) implements a single-sample neural network regression model:

- **Specifications:** Input $x = 1.0$, target $y = 2.0$, hidden activation $h = \tanh(w \cdot x + b)$, output prediction $y_{pred} = v \cdot h + c$, quadratic loss $J = \frac{1}{2}(y_{pred} - y)^2$.
- **Analytical Multivariate Chain Rule:**

$$\delta_{out} = y_{pred} - y, \quad \frac{\partial J}{\partial c} = \delta_{out}, \quad \frac{\partial J}{\partial v} = \delta_{out}\, h$$

$$\delta_{hidden} = \delta_{out}\, v\, (1 - h^2), \quad \frac{\partial J}{\partial w} = \delta_{hidden}\, x, \quad \frac{\partial J}{\partial b} = \delta_{hidden}$$

- **Numerical Verification:** Analytical gradients were verified against two-sided central finite differences ($\varepsilon = 1.0 \times 10^{-6}$):

Parameter (θ)	Analytical Gradient	Central Difference	Absolute Error ($\lvert\Delta\rvert$)	Status
w	`-1.209464475240`	`-1.209464475349`	1.09×10^{-10}	PASSED ($\leq 10^{-5}$)
b	`-1.209464475240`	`-1.209464475349`	1.09×10^{-10}	PASSED ($\leq 10^{-5}$)
v	`-0.710682047486`	`-0.710682047433`	5.31×10^{-11}	PASSED ($\leq 10^{-5}$)
c	`-1.537882842740`	`-1.537882842628`	1.12×10^{-10}	PASSED ($\leq 10^{-5}$)

- **Simultaneous Updates & Convergence:** All 4 parameters are updated simultaneously from immutable prior state: $\theta_{t+1} = \theta_t - \eta \nabla J$. Over 100 training steps ($\eta = 0.1$), loss decreased monotonically from $J_1 = 1.182541818997$ to $J_{100} < 10^{-12}$ ($y_{pred} = 1.999999999768 \approx 2.000000$).

3. Verification & Quality Assurance

The codebase was subjected to exhaustive test-driven verification:

- **CTest Automation:** 7 dedicated test suites compiled under MSVC 19.51.36260.0 in Release mode executed in 0.43 seconds with a 100% pass rate:
- `workbench_core_tests.exe` : 55 assertions passed
- `workbench_geometry_tests.exe` : 72 assertions passed
- `workbench_thermal_tests.exe` : 37 assertions passed
- `workbench_performance_tests.exe` : 80 assertions passed
- `workbench_hardware_tests.exe` : 91 assertions passed
- `workbench_backprop_tests.exe` : 39 assertions passed
- `workbench_ui_tests.exe` : 264 assertions passed
- **Total:** Exactly **638 passing assertions, 0 failed, 0 warnings**.
- **Headless UI Verification:** Edge Puppeteer automation (`scratch/test_stage11a_usability.js`) verified keyboard navigation accessibility, ARIA tab semantics, multi-viewport layout rendering, and contrast tokens across 19 automated assertions.
- **Standalone Package Clean-Directory Validation:** The generated distribution package was extracted and tested in a clean temporary directory (`$env:TEMP\UEW_Standalone_Test`), verifying preflight integrity, WebView2 loader linkage, asset staging, and process shutdown across 32 assertions.

4. Engineering Assumptions & Limitations

1. **Thermal Physics:** Conduction is assumed 1D along the rod axis. Radial heat losses to the environment are neglected ($Bi \ll 0.1$), and material properties (k, A) are assumed temperature-invariant.
2. **Simulated Hardware:** All hardware adapters represent simulated mock instances. Real-world physical hardware communication via WinUSB, FTDI D2XX, or physical JTAG tap state machines is not implemented in this phase.
3. **Neural Model Scope:** The backpropagation module demonstrates fundamental multivariate calculus and simultaneous update mechanics on a single regression sample; it is not intended as a generalized deep learning framework.

4. **Operating System Specialization:** Desktop shell and launcher are specialized for Microsoft Windows 10/11 x64 leveraging native Win32 APIs and WebView2 Evergreen Runtime.

5. Conclusion & Submission Status

The Unified Engineering Workbench successfully fulfills all functional, algorithmic, and packaging requirements of the PASS Science Works R&D Internship 2026–2027 assignment. The native C++17 core provides verified numerical accuracy, high-throughput parallel execution, and robust memory safety. The desktop shell delivers an intuitive, air-gapped user interface without external JavaScript framework dependencies.

Submission Deliverables Summary:

- Standalone Executable Package: `package/UnifiedEngineeringWorkbench-1.0.0-win64.zip` (Verified)
- Test Verification: 638/638 passing assertions (Verified)
- Final Technical Report PDF: `Unified_Engineering_Workbench_Final_Report.pdf` (Generated)
- AI Use Disclosure: `docs/ai_use_disclosure.md` (Complete)
- Third-Party Licensing: `docs/licenses.md` (Complete)
- **Manual Author Actions Remaining:** Attaching the physical handwritten backpropagation appendix and pushing the Git repository to the remote destination.